

EX. NO. 6**IMPLEMENTING FABRIC CHANNELS FOR PRIVACY****AIM**

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE**Step 1: Start Fresh**

Navigate to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Stop any previously running Fabric network.

```
./network.sh down
```

```

. Command not founddev/fabric-samples/test-network [[201~
vboxuser@Ubuntu:~$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 10/10
✓ Container ca_orderer Removed 10.9s
✓ Container ca_org1 Removed 11.6s
✓ Container ca_org2 Removed 11.2s
✓ Container peer0.org1.example.com Removed 2.8s
✓ Container peer0.org2.example.com Removed 3.0s
✓ Container orderer.example.com Removed 2.0s
✓ Volume compose_orderer.example.com Removed 0.1s
✓ Network fabric_test Removed 0.2s
✓ Volume compose_peer0.org1.example.com Removed 0.1s
✓ Volume compose_peer0.org2.example.com Removed 0.1s

```

Start the Fabric network and create the default mychannel.

`./network.sh up createChannel -ca`

```

vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up create
Channel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay
of '3' seconds and using database 'leveldb with crypto from 'Certificate Author
ities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] up 4/4
✓ Network fabric_test Created 0.2s

```

Step 2: Set Peer Environment Variables for Org1

Load the Org1 environment.

`source setOrg1.sh`

If the setOrg1.sh script does not exist, create it using the following commands:

`cat > setOrg1.sh << 'EOF' export PATH=${PWD}/../bin:$PATH export`

`FABRIC_CFG_PATH=${PWD}/../config/ export`

`CORE_PEER_TLS_ENABLED=true export`

`CORE_PEER_LOCALMSPID="Org1MSP"`

`export`

```
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

```
export
```

```
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp export
```

```
CORE_PEER_ADDRESS=localhost:7051
```

```
export
```

```
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem EOF
```

```
source setOrg1.sh
```

Set the TLS certificate path for Org1.

```
export
```

```
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

Set the TLS certificate path for Org2.

```
export
```

```
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ cat > setOrg1.sh << '
OF'
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
export ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
EOF
source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

Step 3: Verify the Existing Channel

List the channels joined by the peer.
peer channel list

The default mychannel should be listed.

```
ts/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-08-31 15:47:27.150 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Channels peers has joined:
mychannel
```

Step 4: Create the Private Channel

Create a second channel named privatechannel.

./network.sh createChannel -c privatechannel

Wait for the command to complete before continuing.

```
channels peers has joined.
mychannel
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh createChannel -c privatechannel
Using docker and docker compose
Creating channel 'privatechannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb'
Network Running Already
Using docker and docker compose
Generating channel genesis block 'privatechannel.block'
Using organization 1
/home/vboxuser/fabric-dev/fabric-samples/test-network/./bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/privatechannel.block -channelID privatechannel
2026-08-31 15:47:56.437 UTC 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-08-31 15:47:56.451 UTC 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-08-31 15:47:56.452 UTC 0003 INFO [common.tools.configtxgen.localconfig] com
```

Step 5: Verify Both Channels

List the channels joined by the peer again.

peer channel list The list should contain:

mychannel privatechannel

If privatechannel is

missing, check the

channel creation

command for errors

before continuing.

Step 6: Deploy Chaincode on the Private Channel Only

Deploy the JavaScript Asset Transfer chaincode to privatechannel.

```
./network.sh deployCC -c privatechannel -ccn basic -ccl javascript -ccp ../asset-transfer-basic/chaincode-javascript
```

The -c privatechannel option ensures that the chaincode is deployed only on privatechannel.

```
Channel 'privatechannel' joined
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-08-31 15:48:42.997 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Channels peers has joined:
mychannel
privatechannel
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -c privatechannel -ccn basic -ccl javascript -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'privatechannel'
executing with the following
- CHANNEL_NAME: privatechannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
```

Step 7: Initialize the Ledger on the Private Channel

Invoke the InitLedger function with endorsements from both Org1 and Org2.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C privatechannel -n basic -peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}'
```

Wait for the transaction to complete successfully before continuing.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C privatechannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}'
2026-08-31 15:50:21.259 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Step 8: Query Assets from the Private Channel

Retrieve all assets stored on privatechannel.

```
peer chaincode query -C privatechannel -n basic -c '{"Args":["GetAllAssets"]}'
```

The initialized assets should be retrieved from the ledger associated with privatechannel.

```
chaincode invoke successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C privatechannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Step 9: Verify Channel Isolation

Attempt to read the same asset from mychannel. peer chaincode query -C

```
mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
```

This query should fail because the basic chaincode was deployed only on privatechannel.

The failure demonstrates that the chaincode and ledger data associated with privatechannel are isolated from mychannel.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
Error: endorsement failure during query. response: status:500 message:"make sure the chaincode basic has been successfully defined on channel mychannel and try again: chaincode basic not found"
```

Step 10: Stop the Fabric Network

Stop and remove the Fabric network after completing the experiment.

```
./network.sh down
```

FLOW DIAGRAM

Start Fabric Network

|



Create mychannel

|



Set Org1 Environment

|



Create privatechannel

|



Verify Both Channels

|



Deploy Chaincode on privatechannel

|



Initialize Private Ledger

|



Query Assets

|



Verify Channel Isolation

|



Stop Network

RESULT

The Hyperledger Fabric network was successfully configured with multiple channels. A separate privatechannel was created and the JavaScript chaincode was deployed only on that channel. The ledger was initialized and its assets were successfully queried. An attempt to access the private channel's chaincode through mychannel failed, demonstrating channel-based ledger isolation and privacy in a permissioned blockchain network.